

Desarrollo de Software en Sistemas Distribuidos

Cátedra: Azcurra – Amaro - Tomczak.

Cuatrimestre: Segundo

Año: 2025

TRABAJO PRÁCTICO

RPC - Empuje solidario

Grupo 5: Integrantes

Nombre	Martin Bombara	Daniel Geier	Sebastián Marioni	Luciano Moliterno
Tareas Realizadas	- Creación de filtro graphql - Guardado de filtro - Presentacion filtros en UI frontend	- Creación de Informe de participación en Eventos Propios (GraphQL) la parte del back. - creación de la rama Informe de participación en eventos propios (GRAPHQL) esta misma se realizó con la ayuda del compañero Luciano - también se creó la rama filtro personalizado del informe de participación en eventos propios (REST)	Consulta de Presidentes y ONGs (SOAP): • Vista y funcionalidad del frontend • Creación del cliente+controller en el backend para consumir y enviar los datos • Método intermediario de Flask que recibe los IDs del frontend y los envía al backend Spring Boot, devolviendo la respuesta al frontend. También termine de realizar las funcionalidades que faltaban del tp 2 (se deja un video mostrando el funcionamiento)	- view login - view registro usuarios - roles usuarios - interfaz proto usuarios - schema db users - db users - login de un usuario se agregó en la rama inventario-grpc-eventos-gateway-darkmode: nuevo microservicio inventario + gateway flask (expone inventario) + docker compose actualizado para levantar inventario y conectar el gateway por gRPC ----- cree las ramas: Gestión de Eventos Externos (Puntos 5 y 6) + Adhesión de Voluntarios a Eventos (Punto 7)

				----- cree la rama: Informes-Donaciones (Puntos 1 y 3) + se ayudó al compañero Daniel con la rama Informe de participación en eventos propios (GRAPHQL)
DNI	40812039	40229425	42281606	40238958

Repositorio Github

<https://github.com/DanyGeier/Grupo-TN-G>

Estrategia de resolución

Tecnologías utilizadas

- Frontend WEB: **TS con React + Vite**
- Cliente Rest <-> gRPC : **Python**
- Servicios backend (Usuarios, Eventos, Donaciones) : **Java con Spring Boot**
- DB Servicio Usuarios y Donaciones: **MySQL**
- DB Servicio Eventos: **MongoDB**
- SMTP Envío de emails: **Mailpit**
- Broker / plataforma para envío de eventos : **Apache Kafka**
- Virtualizacion / Contenedores: **Docker**
- Otras herramientas de desarrollo:
 - **Postman**
 - **Bruno**
 - **Trello**

Descripción de la resolución:

Arquitectura y división de servicios

Para el agregado de informes y reportes en excel sobre los eventos y donaciones del sistema distribuido planteamos un esquema con un nuevo servicio de **informes** el cual expone endpoints GraphQL para realizar las consultas necesarias y almacenamiento de filtros de búsqueda. Para nutrirse de información, lo que hace este nuevo servicio es comunicarse directamente con los servicios ya existentes de inventario y eventos.

De esta forma, mantenemos servicios separados y este servicio nuevo puede consultar en las bases de datos de los otros servicios la información necesaria.

En un caso real, optamos quizás por replicar información de solo lectura para que el servicio de informes sea independiente y se mantenga actualizado a través de Kafka siguiendo con el esquema propuesto de la entrega anterior

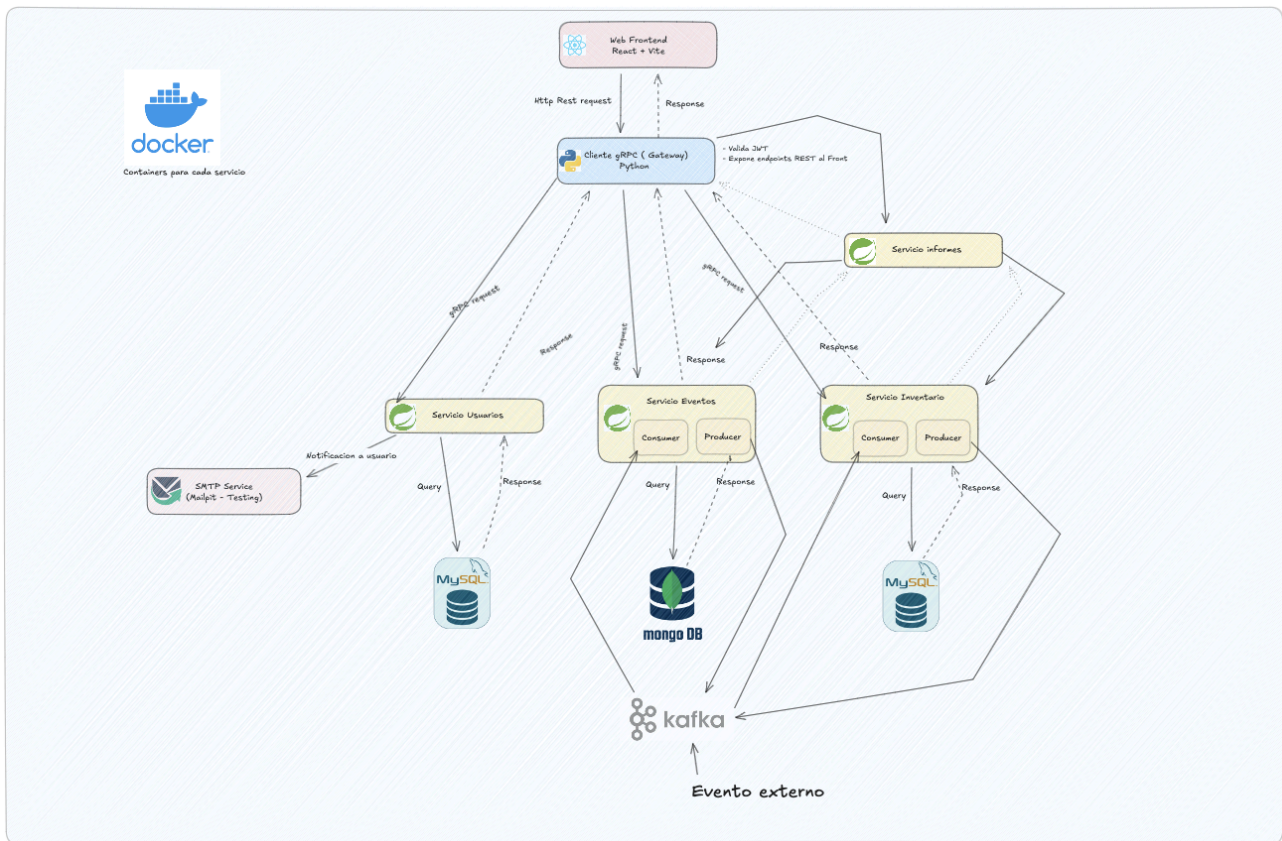
Endpoints rest agregados cuentan con documentación de swagger agregada automáticamente gracias a las annotations que la librería nos provee

Organización de trabajo/división de actividades

Utilizamos Trello para gestionar tareas y coordinar al equipo. Cada proyecto se organiza en tableros con listas que reflejan el estado de las actividades. Las tarjetas representan tareas individuales, y contienen a los miembros responsables de realizarlas.

<https://trello.com/b/yznApxNZ/temp-tp-distribuidos>

Diagramas



Problemas/Errores

- Planteamiento de la solución buscando simplicidad pero manteniendo separación con la idea de trabajar con sistemas distribuidos

Pruebas

Opción 1: Descripción de las pruebas realizadas con las correspondientes capturas de pantalla.

Opción 2: En caso que prefieran hacerlo en video: enlace al video (deberá ser un video con voz, especificando qué pruebas se están ejecutando)

Video de pruebas:

 WebServicesPresentacion.mov

Video TP2 de lo faltante:

 Explicación Frontend-Kafka.mkv